

INFERENCE ENGINE EXTENSION CONTRACT

vLLM-HUST

Plugin Standard 1.0

插件开发与运行标准

本标准规定 vLLM-HUST 运行时插件的最小开发、集成、启动、验证、停止、卸载和回滚合同。

This standard defines the minimum development, integration, startup, verification, shutdown, removal, and rollback contract for a vLLM-HUST runtime plugin.

Document status / 文档状态

This is a stable public engineering standard. Conformance refers to a specific plugin release and its declared compatibility range; it does not imply performance superiority or endorsement of a research claim.

本文件是稳定的公开工程标准。“符合本标准”仅针对某一插件版本及其声明的兼容范围，不代表性能优越，也不构成对研究结论的背书。

Version / 版本	1.0
Status / 状态	Public standard / 公开标准
Applies to / 适用对象	Independently packaged vLLM runtime plugins / 独立封装的 vLLM 运行时插件
Canonical source / 权威源	https://github.com/vLLM-HUST/vllm-hust-website/blob/main/docs/PLUGIN_STANDARD.tex
Project site / 项目网站	https://vllm-hust.sage.org.ai/plugins.html

目录

Document status / 文档状态	1
1 Normative language / 规范性语言	3
2 Scope and boundary / 范围与边界	3
3 Supported entry-point groups / 支持的入口点组	3
4 Package and identity / 包结构与身份	4
5 Registration contract / 注册合同	5
6 Compatibility, configuration, and safety / 兼容、配置与安全	5
7 Build, install, and discovery / 构建、安装与发现	6
8 Enable and start / 启用与启动	6
9 Verification and evidence / 验证与证据	6
10 Stop and disable / 停止与停用	7

11 Remove and roll back / 卸载与回滚	7
12 Conformance requirements / 符合性要求	8
13 Release checklist / 发布检查清单	9
14 References / 参考资料	9

1 Normative language / 规范性语言

The key words **MUST** / **必须**, **SHOULD** / **应当**, and **MAY** / **可以** indicate requirement, recommendation, and permission. Where English and Chinese wording differ, the requirement with the stricter operational safety interpretation applies until the source is corrected.

关键词 **MUST** / **必须**、**SHOULD** / **应当**、**MAY** / **可以** 分别表示强制要求、建议要求和允许事项。中英文出现差异时，在源文件修订前采用运行安全约束更严格的解释。

2 Scope and boundary / 范围与边界

A conforming plugin is an independently versioned Python distribution that extends vLLM through a supported Python entry-point group. It installs without modifying the vLLM-HUST source tree and has a documented enable, verify, disable, remove, and rollback path.

合规插件是独立版本化的 Python 发行包，通过受支持的 Python entry point 扩展 vLLM。安装插件不得修改 vLLM-HUST 源码树，并且必须提供启用、验证、停用、卸载和回滚路径。

Compiler backends, kernel libraries, model conversion, benchmark suites, deployment managers, profilers, and offline analysis tools are adjacent assets unless they register a vLLM runtime entry point. Being used by a plugin does not make an adjacent asset a plugin.

编译器后端、算子库、模型转换工具、基准套件、部署管理器、性能分析器和离线分析工具属于相邻资产，除非它们注册了 vLLM 运行时入口。被插件使用并不会使相邻资产自动成为插件。

Minimum boundary / 最小边界

A plugin **MUST** / **必须** be independently installable, selectable at process startup, identifiable in runtime evidence, and removable without editing the inference-engine source tree. 插件必须能够独立安装、在进程启动时显式选择、在运行证据中可识别，并可在不修改推理引擎源码树的情况下卸载。

3 Supported entry-point groups / 支持的入口点组

Choose the narrowest supported group that matches the extension's responsibility.

Group	Intended use / 用途	Process scope / 进程范围
vllm.general_plugins	Runtime registration and general extensions / 通用运行时注册与扩展	API, engine-core, worker
vllm.platform_plugins	Out-of-tree hardware platform / 树外硬件平台	Relevant initialization processes / 相关初始化进程

Group	Intended use / 用途	Process scope / 进程范围
vllm.io_processor_plugins	Input/output processors / 输入输出处理器	API process
vllm.stat_logger_plugins	Statistics loggers / 统计日志器	Asynchronous API process / 异步 API 进程

vllm.platform_plugins is reserved for hardware-platform implementations. A compiler, kernel library, model-preparation utility, or benchmark is not a platform plugin merely because a runtime uses it.

4 Package and identity / 包结构与身份

```
<plugin-repository>/
|-- pyproject.toml
|-- README.md
|-- CHANGELOG.md
|-- src/<package>/
|   |-- __init__.py
|   `-- plugin.py
`-- tests/
    |-- test_entry_point.py
    |-- test_registration.py
    `-- test_lifecycle.py
```

The package **MUST** / **必须** declare one stable plugin identity. Distribution name, import package, and plugin ID may differ, but all three **MUST** / **必须** remain stable within a release line and be documented together.

```
[project]
name = "<distribution-name>"
version = "<version>"

[project.entry-points."vllm.general_plugins"]
<plugin-id> = "<package>.plugin:register"
```

Every release **MUST** / **必须** record the distribution version, import package, plugin ID, entry-point group, entry callable, supported vLLM range, and tested vLLM-HUST commit.

5 Registration contract / 注册合同

Registration **MUST** / 必须 be re-entrant because vLLM may load plugins in multiple processes and may invoke the same registration path more than once.

```
_REGISTERED = False

def register() -> None:
    global _REGISTERED
    if _REGISTERED:
        return
    # Install one narrow runtime hook.
    _REGISTERED = True
```

A conforming registration function:

- **MUST** / 必须 install the narrowest practical hook and produce the same result on repeated calls;
- **MUST** / 必须 not duplicate patches, threads, sockets, files, or global state;
- **MUST** / 必须 validate required configuration before serving requests;
- **MUST** / 必须 emit one activation record containing plugin ID and version;
- **MUST** / 必须 fail visibly when a required hook or compatibility condition is unavailable;
- **MUST** / 必须 not embed machine paths, ports, device IDs, credentials, or model locations; and
- **SHOULD** / 应当 expose plugin-owned health, counters, or receipts when operationally relevant.

6 Compatibility, configuration, and safety / 兼容、配置与安全

Each release **MUST** / 必须 document:

- supported platform, Python, vLLM, and vLLM-HUST versions;
- incompatible plugins and mutually exclusive runtime modes;
- every plugin-owned configuration variable, type, default, and validation rule;
- required external services or storage and their failure behavior; and
- safe fallback and rollback behavior.

Configuration variables **MUST** / 必须 use a plugin-specific prefix. Secrets **MUST** / 必须 not appear in source, logs, manifests, command examples, or generated artifacts. A plugin **MUST** / 必须 fail closed when invalid configuration could compromise correctness, isolation, or data integrity.

7 Build, install, and discovery / 构建、安装与发现

Build a wheel and install it into the exact Python environment used by every serving process. Deployment-specific values remain external inputs.

```
export BUILD_PYTHON=/path/to/build/python
export RUNTIME_PYTHON=/path/to/serving/python
export PLUGIN_WHEEL=/path/to/plugin.whl

"${BUILD_PYTHON}" -m build
"${RUNTIME_PYTHON}" -m pip install "${PLUGIN_WHEEL}"
```

Before startup, verify discovery using the same runtime interpreter:

```
"${RUNTIME_PYTHON}" -c \
    'from importlib.metadata import entry_points; \
    print([(e.name, e.value) for e in \
    entry_points(group="vllm.general_plugins")])'
```

Installation alone does not prove activation. Multi-node deployments **MUST** / **必须** install and verify the same plugin release in every applicable environment.

8 Enable and start / 启用与启动

Production services **MUST** / **必须** use an explicit allowlist.

```
export PLUGIN_ID=your-plugin-id
export MODEL=your-model
export PORT=your-port

VLLM_PLUGINS="${PLUGIN_ID}" \
    vllm serve "${MODEL}" --port "${PORT}"
```

Multiple plugin IDs are comma-separated. If VLLM_PLUGINS is unset, vLLM loads every discovered plugin. If set to an empty string, vLLM loads none. Service-manager, container, model, port, device, storage, and endpoint values **MUST** / **必须** remain deployment inputs.

9 Verification and evidence / 验证与证据

Activation requires all of the following:

1. the intended entry point is discoverable in every serving environment;
2. registration logs identify plugin ID and version without exposing secrets;
3. the server reaches its readiness endpoint;

4. at least one functional request completes correctly; and
5. plugin-owned health, counters, or receipts are available when declared.

```
curl --fail --silent \
  "http://127.0.0.1:${PORT}/v1/models"
```

Evidence boundary / 证据边界

A smoke test validates discovery and lifecycle only. A performance claim **MUST** / **必须** use a matched baseline and treatment with the same model, request sequence, runtime, topology, and hardware allocation. Evidence **MUST** / **必须** identify whether it is real-online, trace-driven, replay, fixture, simulation, or projected; these labels are not interchangeable.

Smoke 仅验证发现与生命周期。性能声明必须采用模型、请求序列、运行时、拓扑和硬件分配一致的 matched 对照，并准确标记证据性质。

10 Stop and disable / 停止与停用

Plugins are process-scoped. Standard 1.0 does not define hot unload.

1. Gracefully stop the owning service through its supervisor or normal termination signal.
2. Wait for API, engine-core, and worker processes to exit.
3. Confirm plugin-owned threads, sockets, files, shared memory, and device resources are released.
4. Remove the plugin ID from the explicit allowlist, or set `VLLM_PLUGINS=""`.
5. Restart and verify the baseline path.

Do not use broad process-name termination. For a directly owned process whose PID was recorded at startup, graceful termination is:

```
kill -TERM "${SERVER_PID}"
wait "${SERVER_PID}"
```

For supervised deployments, use the supervisor's scoped stop operation rather than signaling an unowned child process.

11 Remove and roll back / 卸载与回滚

Only uninstall after every consuming process has stopped:

```
export PLUGIN_DISTRIBUTION=your-distribution-name
"${RUNTIME_PYTHON}" -m pip uninstall \
  "${PLUGIN_DISTRIBUTION}"
```

Then verify that the entry point is absent, restart the baseline with an explicit allowlist, run the

readiness probe, and retain the previous wheel and configuration until rollback verification is complete. Rollback **MUST** / **必须** restore both software and plugin-owned state to a documented compatible version.

12 Conformance requirements / 符合性要求

A release may be presented as **vLLM-HUST Plugin Standard 1.0 conforming** only when automated tests cover every mandatory item below.

ID	Required evidence	验收证据
C01	Wheel build and clean-environment installation	wheel 构建与干净环境安装
C02	Entry-point discovery in every required process environment	各必要进程环境中的入口点发现
C03	Registration invoked twice without duplicate state	重复注册不产生重复状态
C04	Registration in declared process roles	在声明的进程角色中完成注册
C05	Enabled startup, readiness, and functional request	启用后的启动、就绪与功能请求
C06	Disabled startup, readiness, and baseline request	停用后的启动、就绪与基线路径
C07	Invalid configuration and compatibility rejection	非法配置与不兼容条件拒绝
C08	Graceful shutdown and complete resource cleanup	优雅停止与完整资源清理
C09	Baseline restart after disable or uninstall	停用或卸载后的基线重启
C10	Versioned evidence manifest with software and configuration identity	含软件和配置身份的版本化证据 manifest

The plugin README **MUST** / **必须** contain exact build, install, discover, enable, start, verify, stop, disable, uninstall, and rollback instructions for the released distribution. A conformance statement **SHOULD** / **应当** link to the test run and immutable release artifact.

13 Release checklist / 发布检查清单

Operator handoff / 运维交付

- ☐ Stable package identity and entry point / 稳定的包身份与入口点
- ☐ Compatibility matrix and configuration reference / 兼容矩阵与配置说明
- ☐ Signed or hashed release artifact / 签名或哈希固定的发布产物
- ☐ Explicit enable and disable commands / 明确的启用与停用命令
- ☐ Readiness, correctness, and cleanup evidence / 就绪、正确性与清理证据
- ☐ Rollback artifact and procedure / 回滚产物与操作流程
- ☐ No credentials or deployment-specific constants / 不含凭据或部署专用常量

14 References / 参考资料

- vLLM Plugin System
- Python Packaging Entry Points Specification
- vLLM-HUST Plugin Ecosystem